CS 5402: Introduction to Data Mining

SP2026

(Lecture - 17)

Long Short-Term Memory (LSTM)

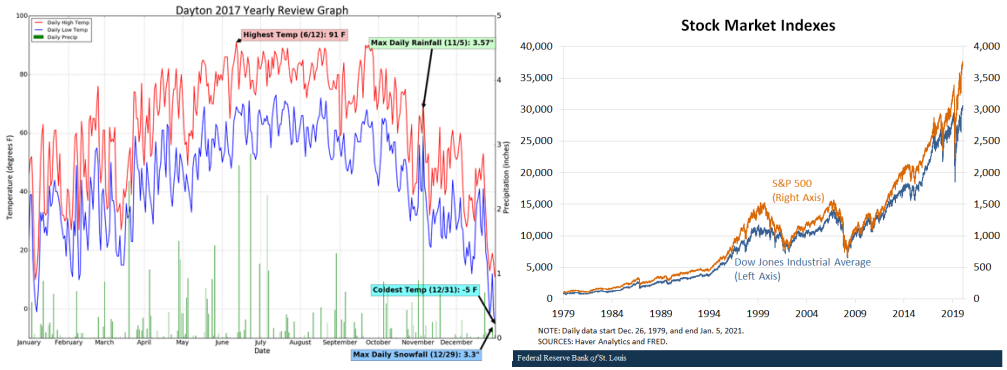
Dr. Huiyuan Yang

Department of Computer Science
Missouri S&T

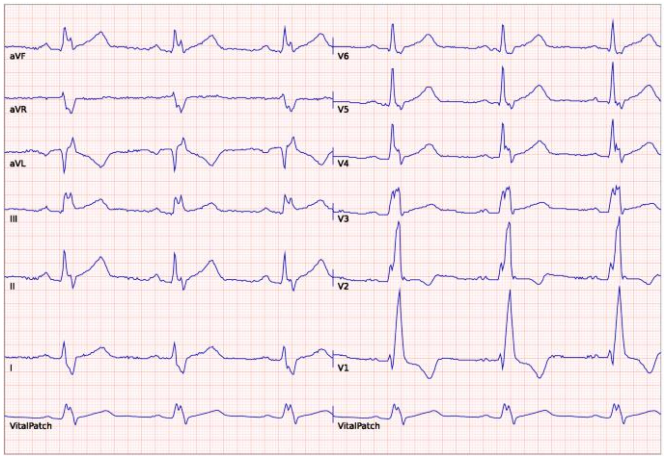
https://hyang428.github.io

MISSOURI S&T

Introduction: Sequential data (Examples)



Time series (e.g., stock prices, weather data)



Sensor signals



Video frames

"I took CS5402 in Spring 2026"

Natural language (e.g., text, speech)

Introduction: Sequential data (key characteristics)

Key Characteristics:

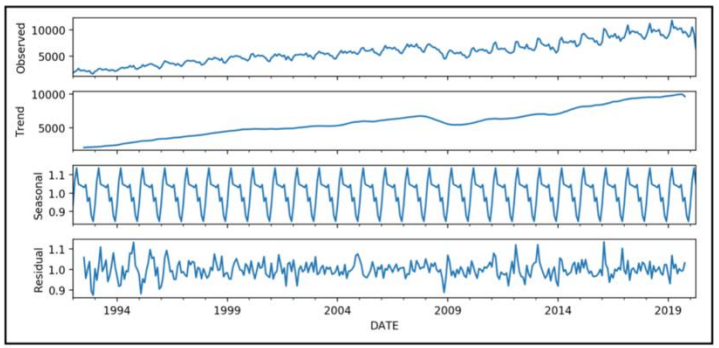
•**Order matters**: Changing the order of elements changes the meaning.

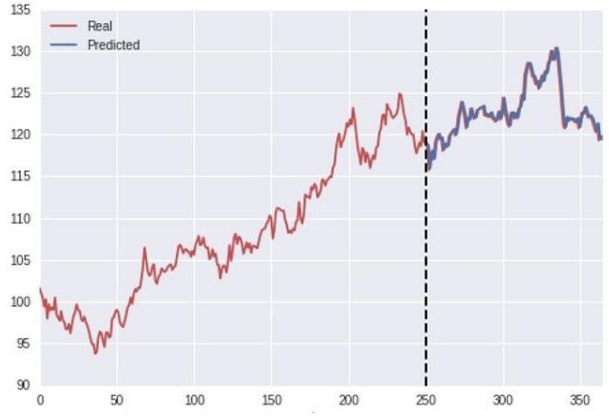
The dog chased the cat. v.s. The cat chased the dog.

•**Contextual dependency**: Later elements often depend on earlier ones.

I am very hungry, and I can eat _____ {"a lot", "nothing"}

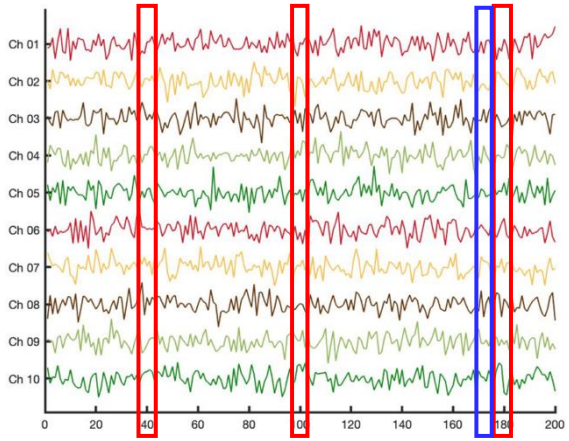
•**Time or position** is often a crucial component.

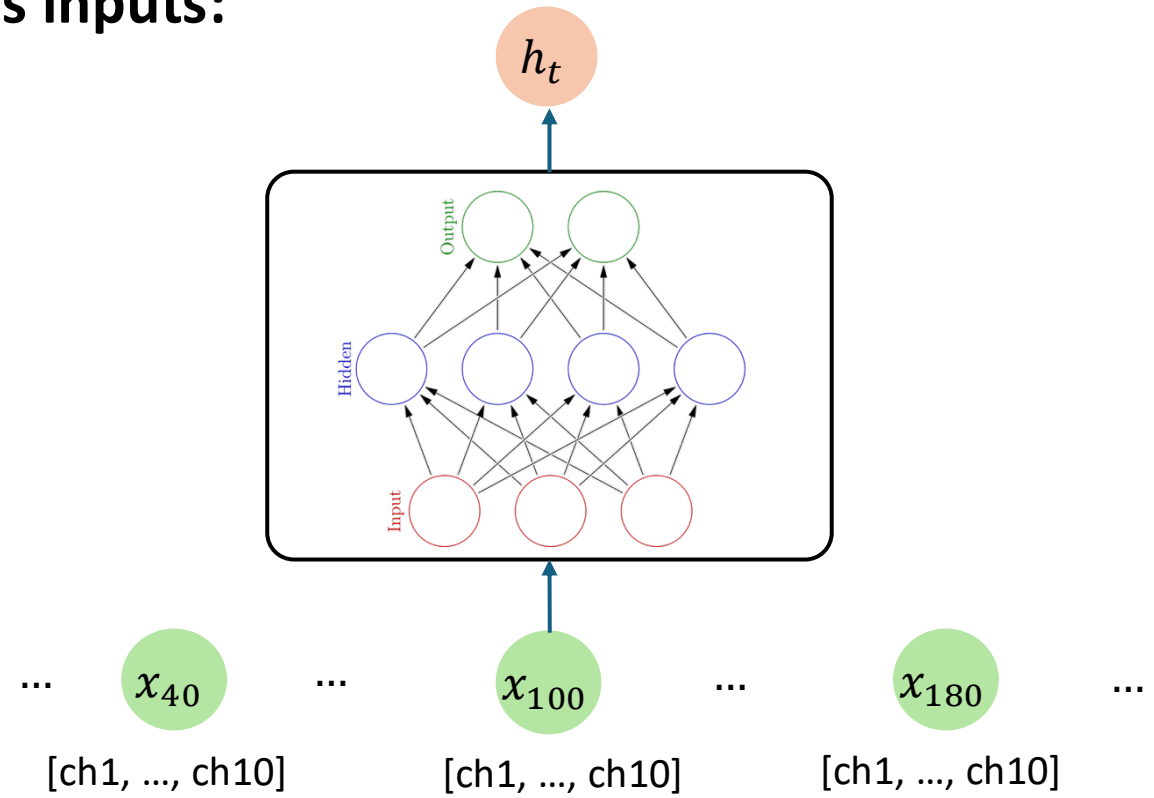




Why traditional neural networks (NNs) struggle with sequence data?

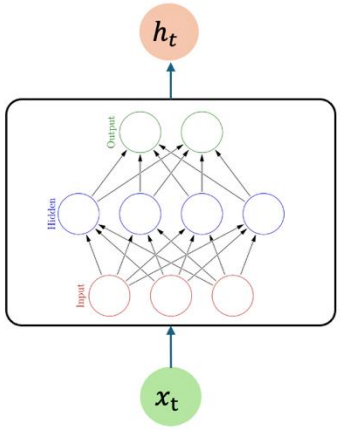
- **Fixed input and output size:**
- **No memory of previous inputs:**



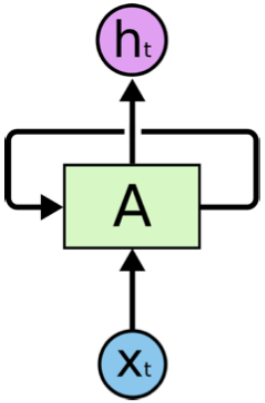


Recurrent Neural Network (RNN)

- Feed forward Network (FFN) :
  - Information flows only in the forward direction. **No cycles or Loops**
  - Decisions are based on current input, **no memory** about the past
  - Doesn't know how to handle sequential data

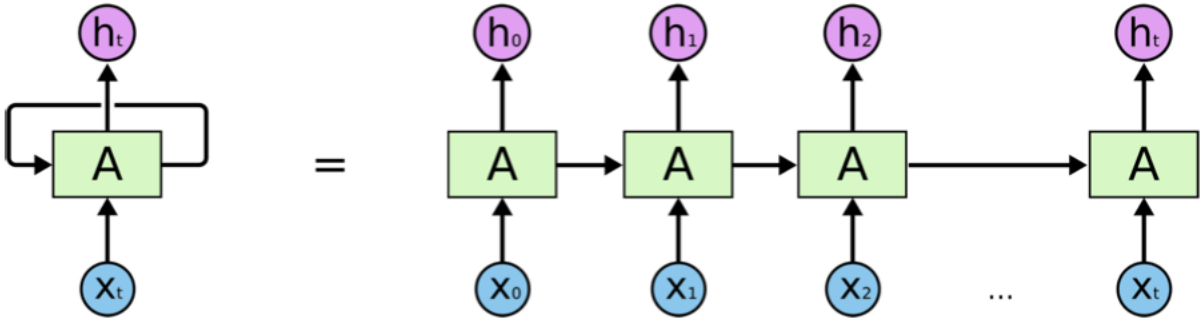


- Solution to FFN : Recurrent Neural Network
  - Can handle sequential data
  - Considers the current input and also the previously received inputs
  - Can memorize previous inputs due to its internal memory



Recurrent Neural Network (1)

- RNN

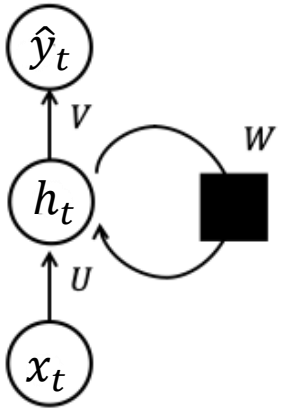


An unrolled recurrent neural network

- Useful in a variety of problems:
  - Speech recognition
  - Image captioning
  - Translation
  - etc

Recurrent Neural Network (2)

- Math behind RNN:



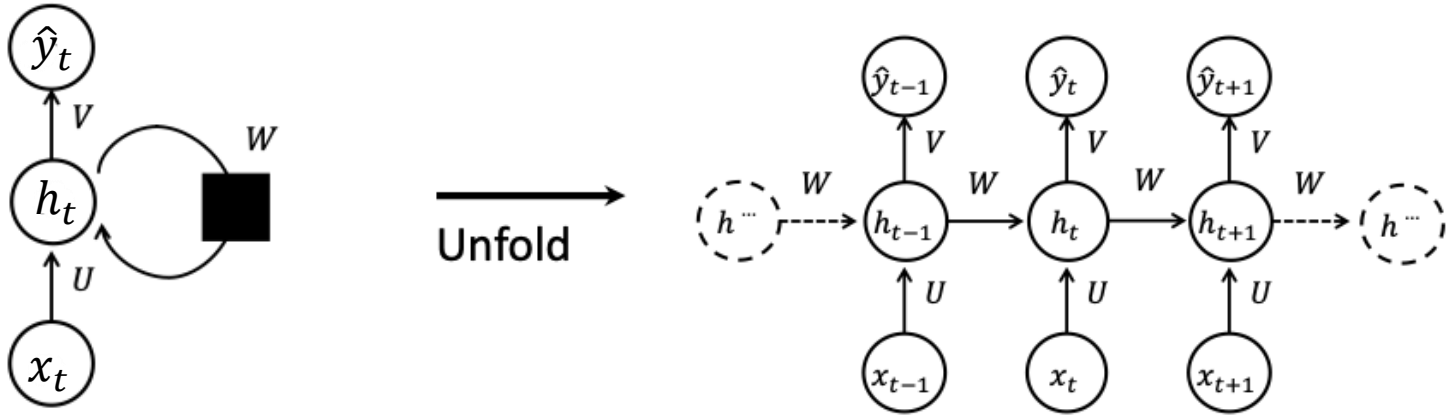
$$h_t = \tanh(Ux_t + Wh_{t-1})$$

$$\hat{y} = f(Vh_t)$$

- h_t: hidden state at time step t
- x_t: input at time step t
- U, W, V: weight matrices. Filters (U, W) that determine how much importance to accord to both the present input and the past hidden state. V is output transformation matrix.

Recurrent Neural Network (3)

- Math behind RNN:



$$h_t = \tanh(Ux_t + Wh_{t-1})$$

$$\hat{y} = f(Vh_t)$$

Parameters U, V, W are shared across all time steps.

Recurrent Neural Network (4)

Make a prediction:

- Let's see how it makes a prediction
- In the beginning, initial hidden state h_0 is filled with zero or random value
- Also we assume the model is already trained. (we will see how it is trained soon)



Recurrent Neural Network (5)

Make a prediction:

- Assume we currently have observation x_1 and want to predict x_2
- We compute hidden states h_1 first

$$h_1 = \tanh(Ux_1 + Wh_0)$$

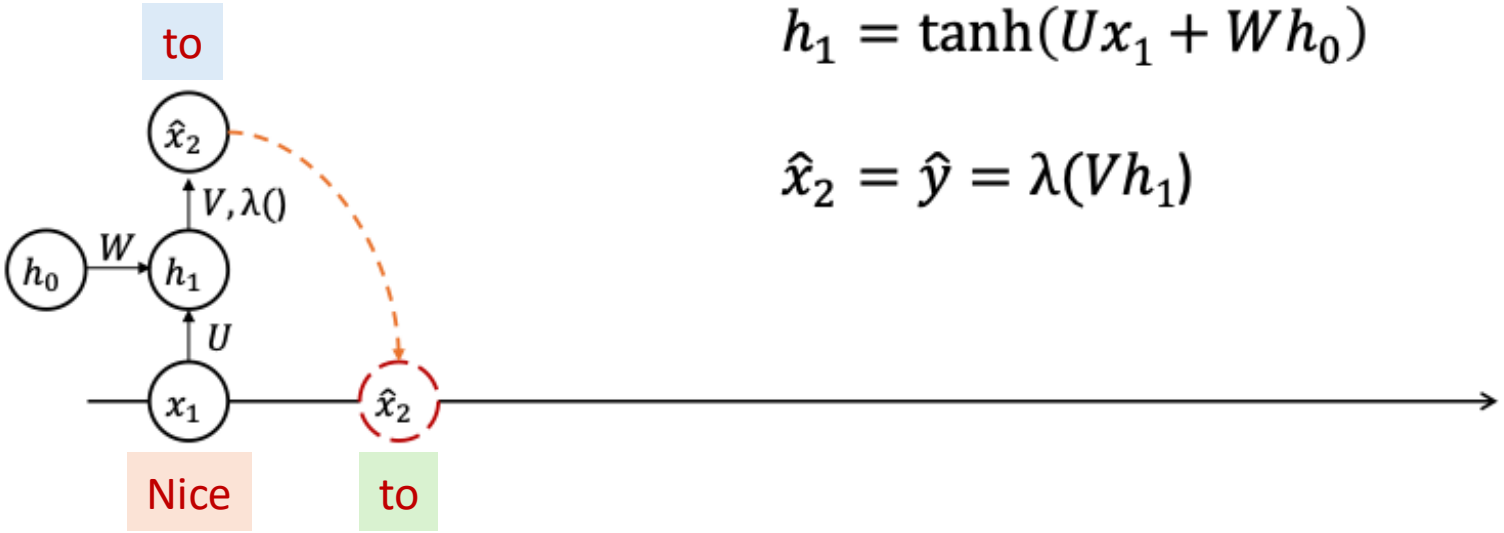


Nice

Recurrent Neural Network (6)

Make a prediction:

- Then we generate prediction:
- Vh_1 is a real-valued vector or scalar value (depends on the size of output matrix V)



$$h_1 = \tanh(Ux_1 + Wh_0)$$

$$\hat{x}_2 = \hat{y} = \lambda(Vh_1)$$

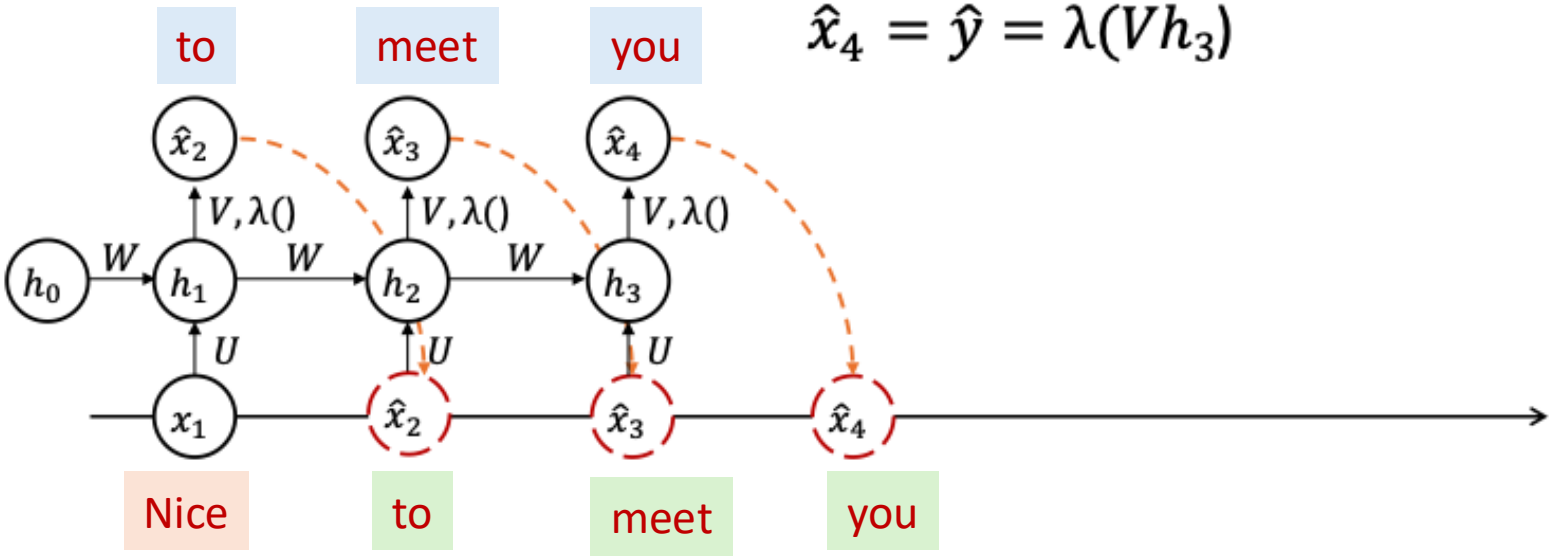
Recurrent Neural Network (7)

Make a prediction:

- Same mechanism applies forward in time..

$$h_3 = \tanh(U\hat{x}_3 + Wh_2)$$

$$\hat{x}_4 = \hat{y} = \lambda(Vh_3)$$

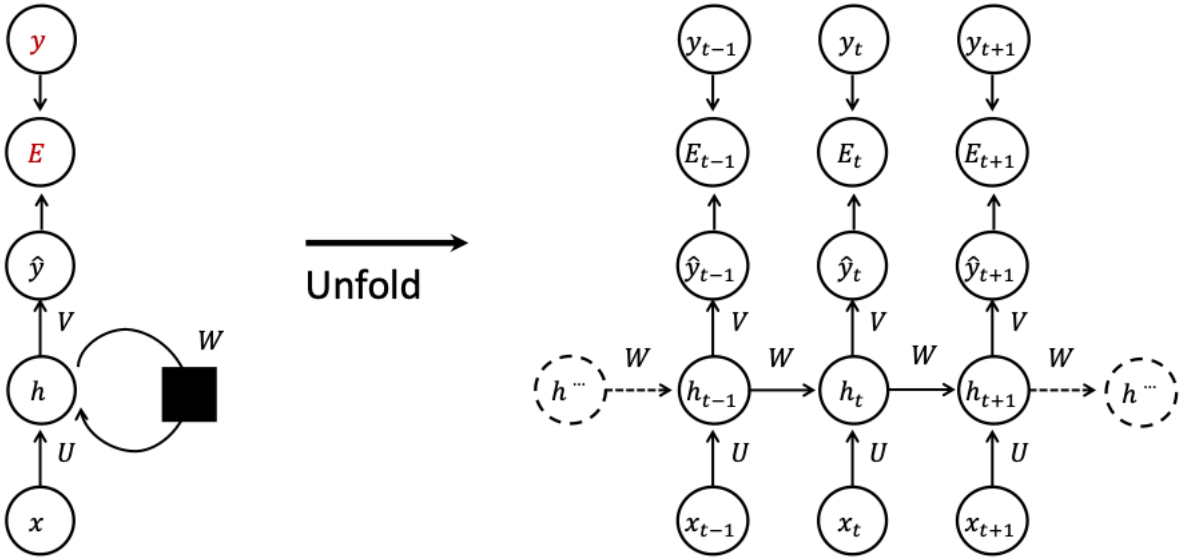


How to train RNN? (1)

Before make train happen, we need to define these:

- y_t: true target
- $\hat{y}_t$: output of RNN
- E_t: loss (error), difference between the true target and output.

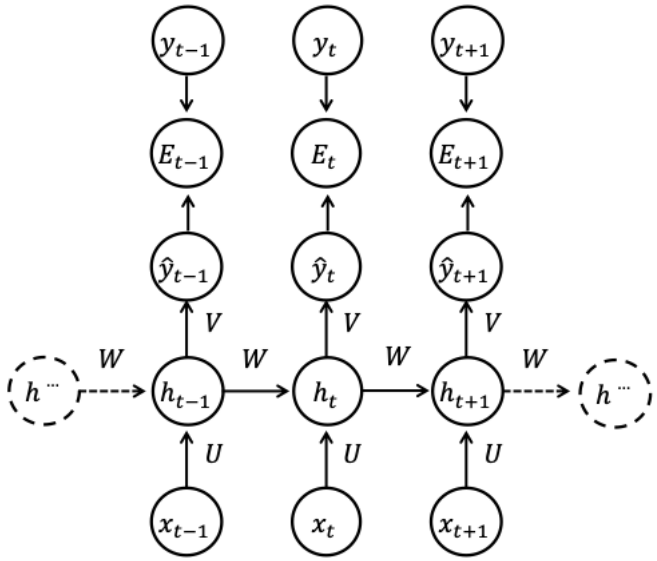
With the loss, the RNN will be like:



How to train RNN? (2)

Backward Propagation Through Time (BPTT)

- Extension of standard backpropagation that performs gradient descent on an unfolded network
- Goal is to calculate gradients of the error with respect to parameters U, V, and W, and learn desired parameters using Stochastic Gradient Descent



How to train RNN? (3)

Backward Propagation Through Time (BPTT)

- To update in one training example (sequence), we sum up the gradients at each time of the sequence:

$$h_t = \tanh(Ux_t + Wh_{t-1})$$

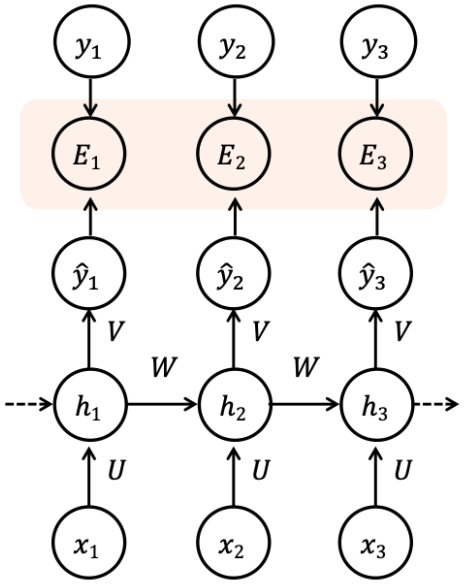
$$\hat{y} = f(Vh_t)$$

$h_0 \xrightarrow{W} h_1 \xrightarrow{W} h_2 \xrightarrow{W} \cdots \xrightarrow{W} h_t$

h_t depends on all previous hidden states due to recurrence. → chain rule

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}.$$

$$= \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial E_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W}.$$



Issues: Vanishing and Exploding Gradient Problem

- In RNN, we repeatedly multiply W along with a input sequence:

$$h_t = \tanh(Ux_t + Wh_{t-1})$$

- The recurrence multiplication can result in difficulties called exploding and vanishing gradient problem.

For example, we can think of simple RNN without inputs x, then:

$$h_t = Wh_{t-1}$$

It can be simplified to:

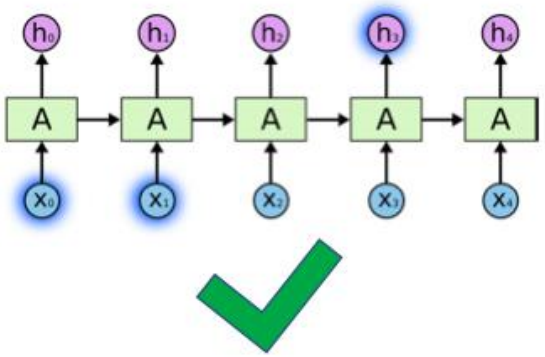
$$h_t = (W^t)h_0 = Adiag(\lambda^t)A^{-1}h_o$$

Any eigenvalues λ_i that are not near an absolute value of 1 will either:

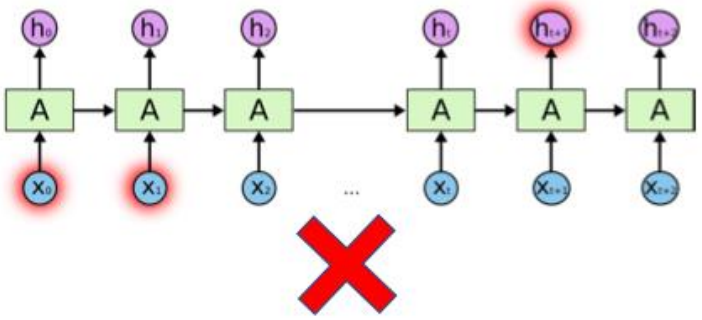
- Explode if they are greater than 1 in magnitude
- Vanish if they are less than 1 in magnitude.

Issues: Long Distance Dependencies

- A small example where RNN can work perfectly:
  - Prediction of the last word in the sentence: “the clouds are in the sky”
- It is very difficult to train RNN to retain information over many time steps
- RNN can’t handle situation where the gap between the relevant information and the point where it is needed is very large.



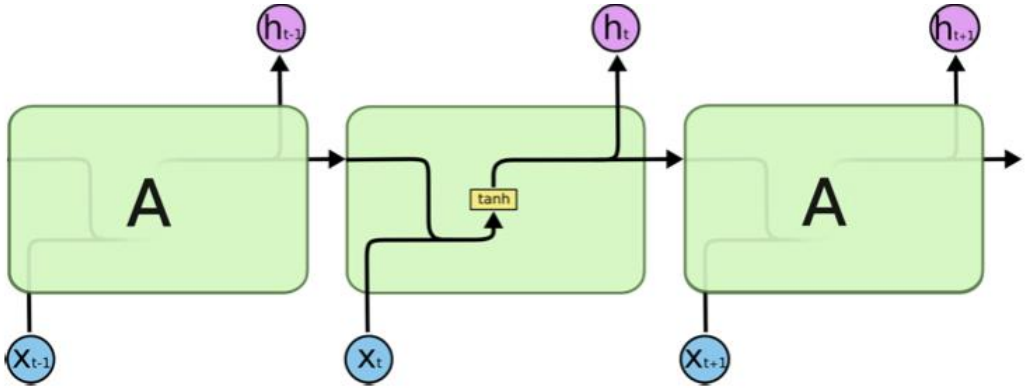
Short sequence:



Long sequence:

Long Short-Term Memory (LSTM)

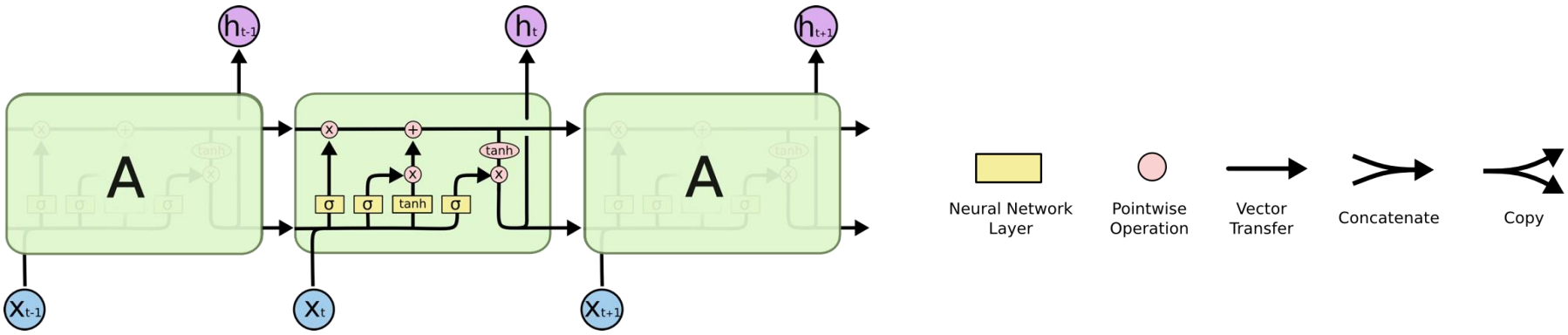
- **Long Short Term Memory networks** – usually just called "**LSTMs**" – are a special kind of RNN, capable of learning **long-term dependencies**. *Hochreiter & Schmidhuber (1997)*
- Capable of modeling longer term dependencies by having memory cells and gates that controls the information flow along with the memory cells.



- Prevents vanishing/exploding gradient problem and allows network to retain state information over longer periods of time.

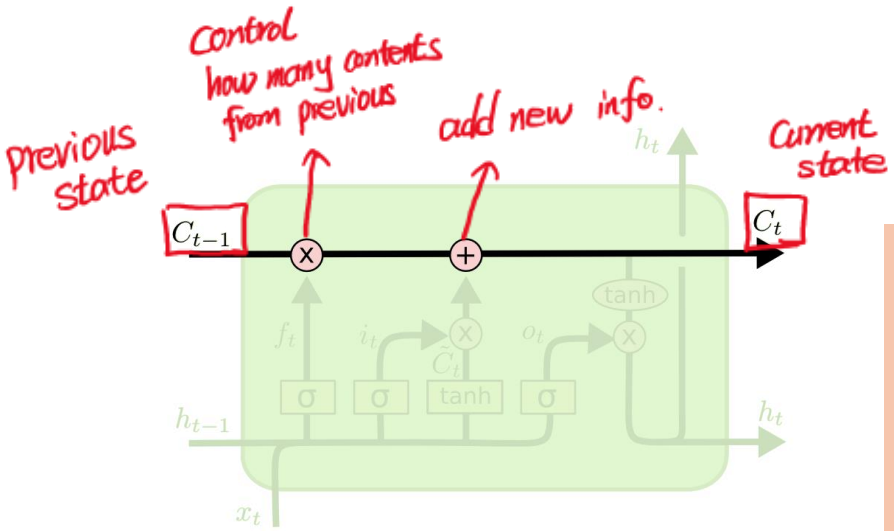
LSTM Network Architecture

- **LSTM** have the same chain-like structure except for the repeating module.
- The contents of the memory cells C" are regulated by various gates:
  - Forget gate f_t
  - Input gate i_t
  - Output gate o_t
- Each gates are composed of affine transformation with Sigmoid activation function



Cell State

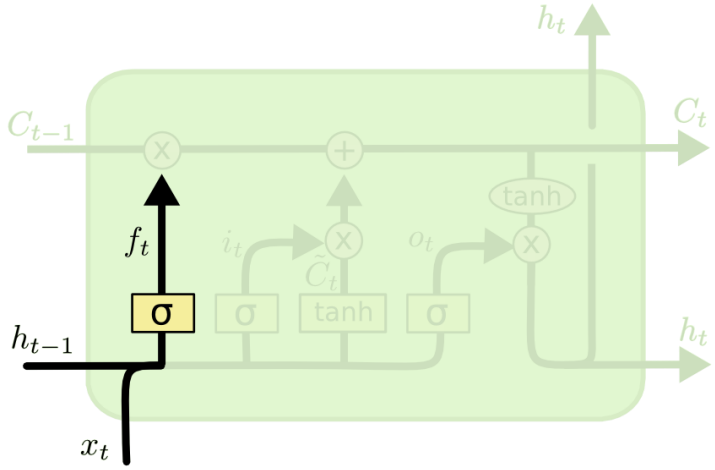
- The key to LSTMs is the **cell state**, the horizontal line running through the top of the diagram.
- It acts like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to flow along it unchanged.
- Maintains a vector C_t that is the same dimensionality as the hidden state, h_t
- Information can be added or deleted from this state vector via the forget and input gates.



Analogy:
Like reading a book and keeping the main character's name in memory even as chapters pass.

Forget Gate

- It determines how many contents from previous cell C_{t-1} will be erased.
- Forget gate computes a 0-1 value using a sigmoid output function from the input x_t, and the current hidden state h_t:
- The sigmoid generates values between 0 and 1:
  - 0: completely remove information in the dimension
  - 1: completely keep information in the dimension



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

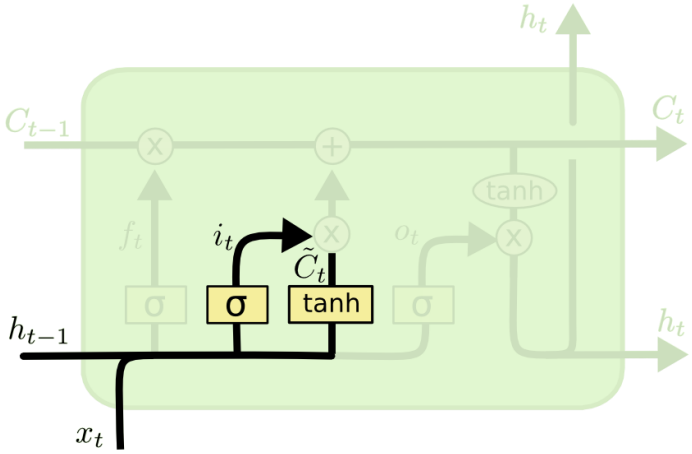
activation function

mlp.

Like erasing irrelevant parts of your memory.

Input Gate

- New candidate cell states $\tilde{C}_t$ are created as a function of h_{t-1} and x_t
- Input gates i_t decides how much of values of the new candidate cell states $\tilde{C}_t$ are combined into the cell states.
- Decides what new information we're going to store in the cell state. Updates the memory.



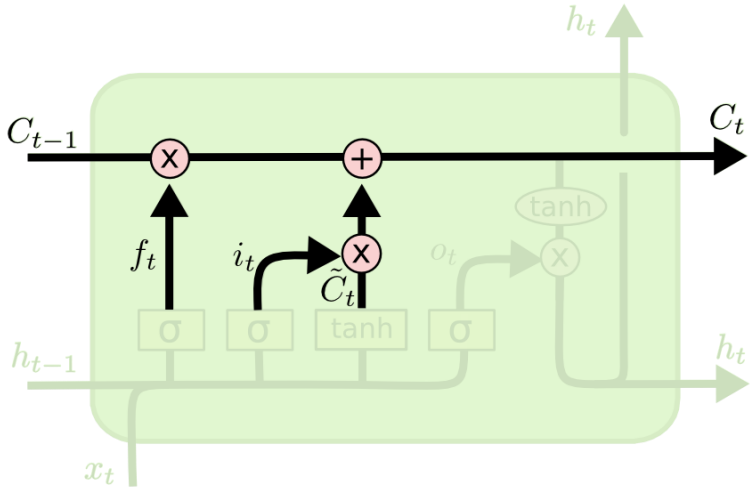
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Like writing down new information in a notebook.

Updating the Cell State

- The previous cell states C_{t-1} are updated to the new cell states C_t by using the input and forget gates with new candidate cell states.



$$C_t = f_t \otimes C_{t-1} + i_t \otimes \tilde{C}_t$$

forget gate — input gate

previous cell states — new cell candidate

Why LSTM reduces exploding gradients (mathematical view)

Step 1: Look at gradient flow

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

Now expand over time:

$$\frac{\partial c_T}{\partial c_1} = \prod_{t=2}^{T} f_t$$

Key difference from RNN:

•RNN: repeated multiplication by large matrices $W_h^T W_h^T \dots$

•LSTM: gates are bounded (sigmoid), $f_t \in (0, 1)$, prevents uncontrolled growth

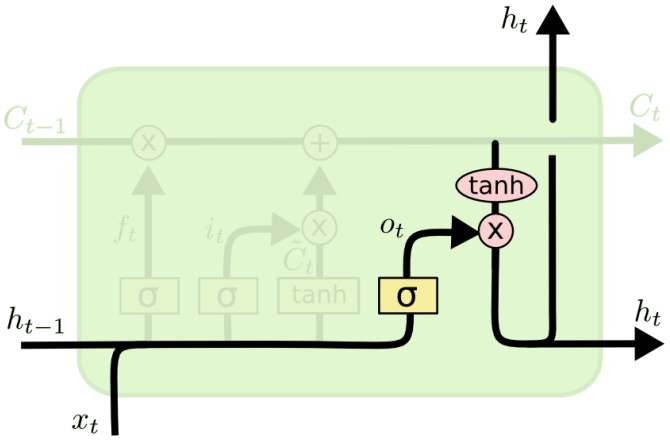
f_t is **learned and controlled**, and model can make:

$f_t \approx 1$ →gradient preserved

$f_t \approx 0$ →forget irrelevant info

Output Gate

- Output will be based on cell state C_t with filter from output gate o_t
- The output gate o_t decides which part of cell state C_t will be in the output.
- Then the final output is generated from tanh-ed cell states filtered by o_t:



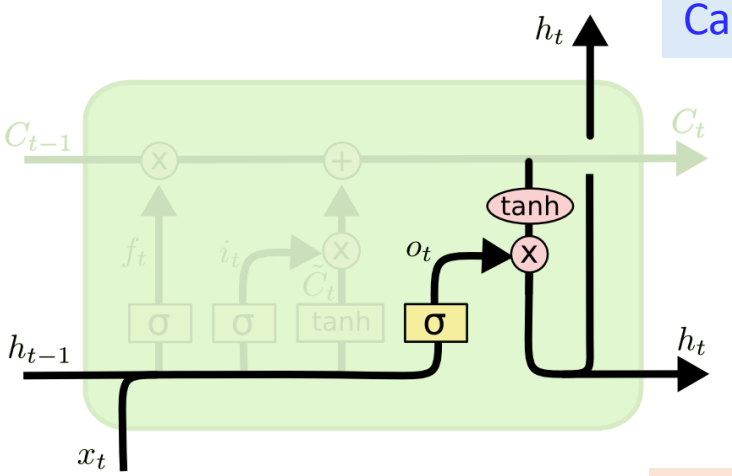
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Like deciding what part of your notes to share with someone else.

LSTM: Hidden state vs. Cell state

Why do we need both a hidden state h_t and a cell state C_t?



Carries **long-term** memory across many time steps

"long-term memory" "trends"

Carries **short-term** information used for immediate output or next layer

"your thoughts right now" "responsive to short-term signals"

LSTM Training

- Trainable with backprop derivatives such as:
  - Stochastic gradient descent (randomize order of examples in each epoch) with momentum (bias weight changes to continue in same direction as last update).
  - ADAM optimizer (Kingma & Ma, 2015)
- Each cell has many parameters (W_f, W_i, W_C, W_o)
  - Generally requires lots of training data.
  - Requires lots of compute time that exploits GPU clusters.
- The good news:
  - You don't have to worry about all those internal details when using libraries such as **Pytorch**, **Tensorflow**.

In-Class Participation Question: Q8

1. **[True] [False]** LSTM networks can process sequences of variable lengths.
2. **[True] [False]** LSTM cells are only useful for natural language processing tasks.
3. **[True] [False]** The vanishing gradient problem is completely solved by LSTM networks.
4. **[True] [False]** The forget gate in an LSTM cell decides what information should be removed from the cell state.
5. **[True] [False]** LSTM networks use only one type of memory to store information across time steps.

Question: Q12

1. **[True] [False]** LSTM networks can process sequences of variable lengths.

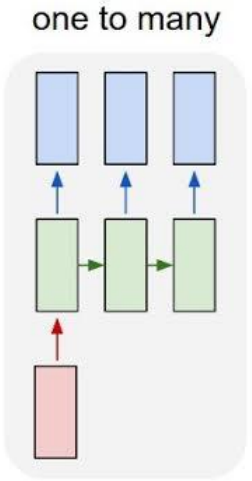
2. **[True] [False]** LSTM cells are only useful for natural language processing tasks.

3. **[True] [False]** The vanishing gradient problem is completely solved by LSTM networks.

4. **[True] [False]** The forget gate in an LSTM cell decides what information should be removed from the cell state.

5. **[True] [False]** LSTM networks use only one type of memory to store information across time steps.

Example of LSTM Applications: One-to-Many



- Input: non-sequence vector
- Output: sequence of vectors

- After the first time step, hidden states are updated with only previous step's hidden states.
- Example: **Sentence generation given image**
  - Typically the input image is processed with CNN to generate a real-valued vector representation
  - During training, true target is a sentence (sequence of words) about the training image



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



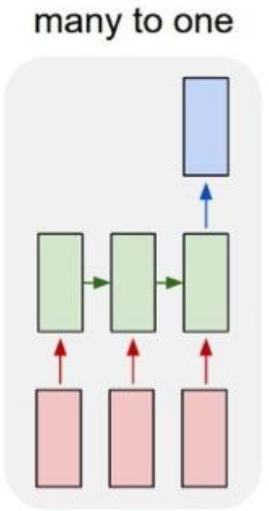
A tennis player in action on the court



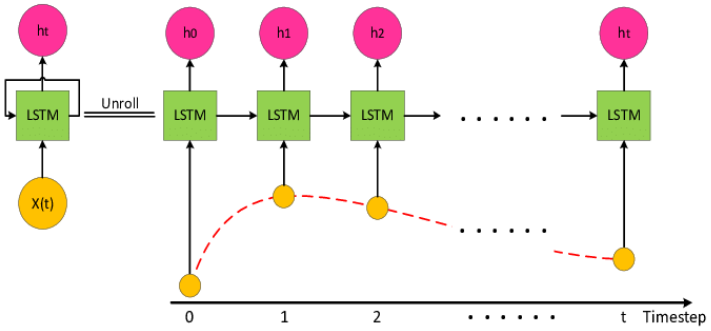
Two giraffes standing in a grassy field

Image Captioning

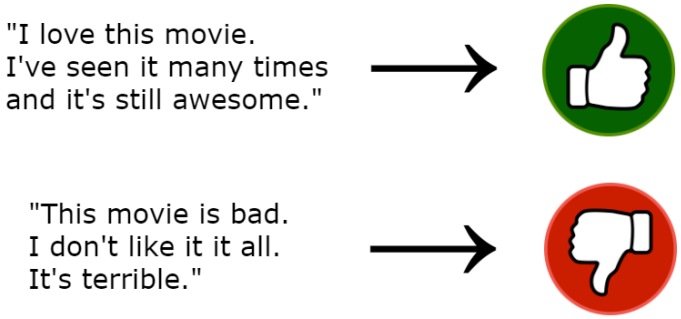
Example of LSTM Applications: Many-to-One



- Input: sequence of vectors
- Output: non-sequence vector
- Only the last time step's hidden states is used as the output
- Example:
  - **Sequence classification**
  - sentiment classification

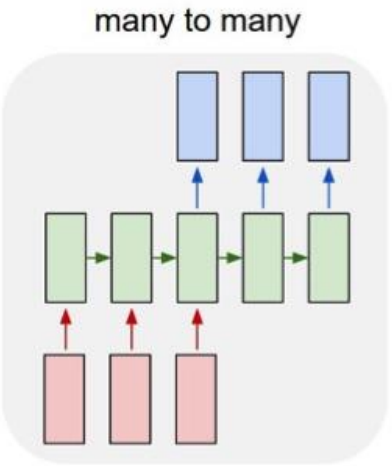


Time series classification

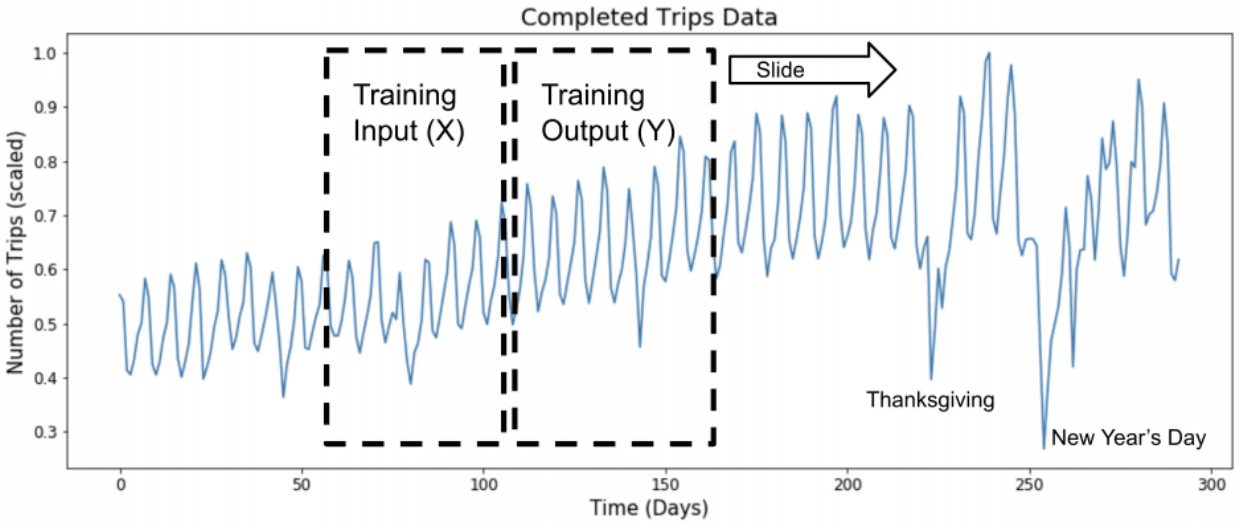


Sentiment classification

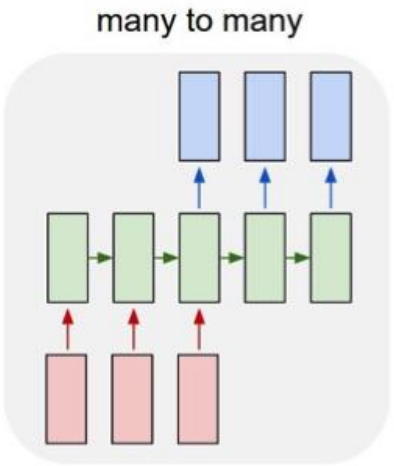
Example of LSTM Applications: Many-to-Many (Time Series)



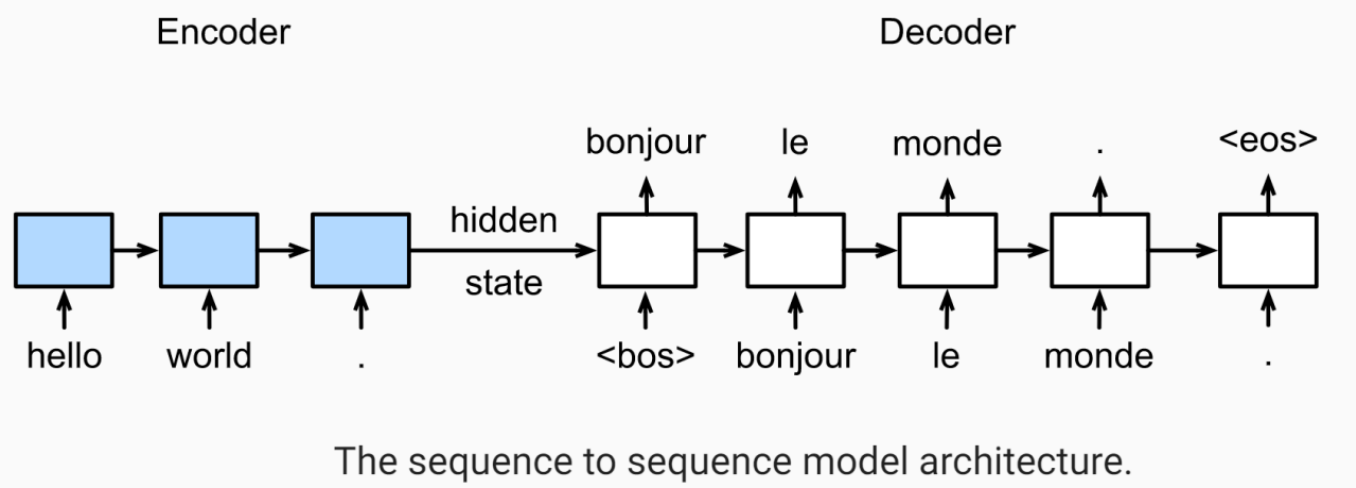
- Input: sequence of vectors
- Output: sequence of vectors
- Generate a sequence given another sequence
- Example: **Time Series Forcasting**
  - They work by taking a sequence of past observations (e.g., the last 5 hours) and using them to predict the next value in the sequence (e.g., the temperature in the 6th hour).



Example of LSTM Applications: Many-to-Many (Text Translation)



- Input: sequence of vectors
- Output: sequence of vectors
- Generate a sequence given another sequence
- Example: **Machine translation**
  - Especially parameterized by what is called "Encoder-Decoder" model



The sequence to sequence model architecture.

Bidirectional RNN (1)

- In some applications, such as speech recognition or machine translation, dependencies over time not only lie in forward in time but also lie in backward in time

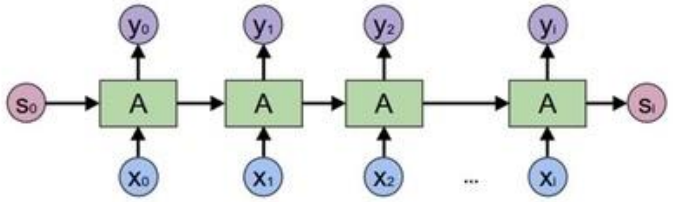
  - I am very hungry, and I can eat _____ {”a lot”, “nothing”}

  - I am ___ hungry, and I can eat half a pig. {”very”, “not”}

Bidirectional RNN (2)

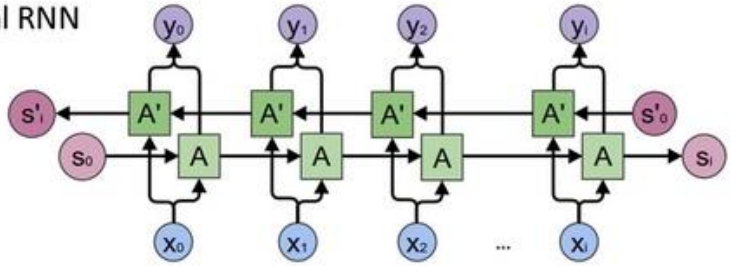
- In some applications, such as speech recognition or machine translation, dependencies over time not only lie in forward in time but also lie in backward in time
- To model these, two RNNs are trained together forward RNN and backward RNN
- Each time step's hidden states from both RNNs are concatenated to form a final output

RNN



Forward $\overrightarrow{h_t} = \text{LSTM}\left(x_t, \overrightarrow{h_{t-1}}\right)$

Bi-directional RNN



Forward $\overrightarrow{h_t} = \text{LSTM}\left(x_t, \overrightarrow{h_{t-1}}\right)$

Backward $\overleftarrow{h_t} = \text{LSTM}\left(x_t, \overleftarrow{h_{t+1}}\right)$

train two LSTMs at the same time (share inputs, but not parameters)

Example: BiLSTM

Example: NLP: Sentence -> Class

"I love this movie" -> Positive

1. I -> love -> this -> movie. (Forward)
2. I <- love <- this <- movie. (Backward)
3. Concatenate hidden states $h_t = \left[\overrightarrow{h_t}; \overleftarrow{h_t}\right]$
4. Output prediction (Positive)

Can we use BiLSTM for real time speech recognition?

Key Takeaways

- **Solves Vanishing Gradient:** Allows learning of long-term dependencies.
- **The Cell State:** Acts as a highway for information to flow unchanged.
- **Gating Mechanism:** Forget, Input, and Output gates regulate the flow of information precisely.
- **Versatile:** The standard for most sequence-based tasks until the rise of Transformers.

After-Class Questions

- Q1: Explain the roles of the following gates in LSTM: Forget gate, Input gate, and Output gate.
- Q2: Why can LSTM alleviate the vanishing gradient problem?
- Q3: Compare RNN and LSTM in terms of gradient propagation.
- Q4: What happens if the forget gate is always close to 1?
- Q5: In what types of tasks is LSTM more suitable than CNN? Why?